

**School of Computer Science and Artificial Intelligence****Lab Assignment # 21.5**

**Program** : B. Tech (CSE)  
**Specialization** :  
**Course Title** : AI Assisted coding  
**Course Code** :  
**Semester** II  
**Academic Session** : 2025-2026  
**Name of Student** : K. Deepak  
**Enrollment No. :** 2303A51T02  
**Batch No.** : 52  
**Date** : 10-04-2026

**Task A – AI-Based Initial Program Development:**

**Prompt:**Generate a simple Python program for a Student Grade Calculator using marks input.

**Code & Output:**

```
2 # Generate a simple Python program for a Student Grade Calculator using marks input.
3 def calculate_grade(marks):
4     if marks >= 90:
5         return 'A'
6     elif marks >= 80:
7         return 'B'
8     elif marks >= 70:
9         return 'C'
10    elif marks >= 60:
11        return 'D'
12    else:
13        return 'F'
14 # Get user input for marks
15 try:
16     marks = float(input("Enter the marks obtained: "))
17     grade = calculate_grade(marks)
18     print(f"The grade for marks {marks} is: {grade}")
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
● akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Enter the marks obtained: 89
The grade for marks 89.0 is: B
● akash@AKASHs-MacBook-Air ai_assis % source /Users/akash/Desktop/ai_assis/.venv/bin/activate
○ (.venv) akash@AKASHs-MacBook-Air ai_assis %
```

**Explanation:**

- Takes marks as input
- Uses if-elif-else conditions
- Prints grade based on range

**Task B – AI-Assisted Application Improvement:**

**Promot:**Enhance the Student Grade Calculator by adding input validation, multiple subjects, and average calculation.

## Code & Output:

```
lab_21.5.py > ...
22 # -----Task B - AI-Assisted Application Improvement
23 # Enhance the Student Grade Calculator by adding input validation, multiple subjects, and average calculation.
24 def calculate_grade(marks):
25     if marks >= 90:
26         return 'A'
27     elif marks >= 80:
28         return 'B'
29     elif marks >= 70:
30         return 'C'
31     elif marks >= 60:
32         return 'D'
33     else:
34         return 'F'
35 def get_marks(subject):
36     while True:
37         try:
38             marks = float(input(f"Enter the marks obtained in {subject}: "))
39             if 0 <= marks <= 100:
40                 return marks
41             else:
42                 print("Marks should be between 0 and 100. Please try again.")
43         except ValueError:
44             print("Please enter a valid number for marks.")
45 subjects = ['Math', 'Science', 'English']
46 total_marks = 0
47 for subject in subjects:
48     marks = get_marks(subject)
49     total_marks += marks
50     grade = calculate_grade(marks)
51     print(f"The grade for {subject} with marks {marks} is: {grade}")
52 average_marks = total_marks / len(subjects)
53 average_grade = calculate_grade(average_marks)
54 print(f"The average marks are: {average_marks}")
55 print(f"The average grade is: {average_grade}")
56
57
58
```

```
7/Users/akash/Desktop/ai_assis/.venv/bin/python 7/Users/akash/Desktop/ai_assis/lab_21.5.py
● akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Enter the marks obtained in Math: 90
The grade for Math with marks 90.0 is: A
Enter the marks obtained in Science: 87
The grade for Science with marks 87.0 is: B
Enter the marks obtained in English: 76
The grade for English with marks 76.0 is: C
The average marks are: 84.33333333333333
The average grade is: B
● akash@AKASHs-MacBook-Air ai_assis % source /Users/akash/Desktop/ai_assis/.venv/bin/activate
○ (.venv) akash@AKASHs-MacBook-Air ai_assis %
```

### # Explanation:

# 1. The `calculate\_grade` function remains unchanged and is used to determine the grade based on the marks.

# 2. The `get\_marks` function is added to handle input validation for each subject, ensuring that the marks entered are between 0 and 100.

# 3. A list of subjects is defined, and a loop is used to get marks for each subject, calculate the grade, and accumulate the total marks.

# 4. After collecting marks for all subjects, the average marks and average grade are calculated and displayed to the user.

## Task C – Git Workflow Practice:

**Prompt:** Give step-by-step Git commands to initialize repo, create branch, commit changes, and merge.

**Output:**

```

1 # -----Task C - Git Workflow Practice
2 # Give step-by-step Git commands to initialize repo, create branch, commit changes, and merge.
3 # 1. Initialize a new Git repository:
4 # git init
5 # 2. Create a new branch for development:
6 # git checkout -b feature/student-grade-calculator
7 # 3. Add the Python file to staging:
8 # git add lab_21.5.py
9 # 4. Commit the changes with a message:
10 # git commit -m "Add Student Grade Calculator with input validation and average calculation"
11 # 5. Switch back to the main branch:
12 # git checkout main
13 # 6. Merge the feature branch into main:
14 # git merge feature/student-grade-calculator
15 # 7. (Optional) Push the changes to a remote repository:
16 # git push origin main
17
18

```

## # Explanation:

- # 1. The `git init` command initializes a new Git repository in the current directory.
- # 2. The `git checkout -b` command creates and switches to a new branch named `feature/student-grade-calculator`.
- # 3. The `git add` command stages the changes made to the `lab\_21.5.py` file for the next commit.
- # 4. The `git commit` command commits the staged changes with a descriptive message.
- # 5. The `git checkout main` command switches back to the main branch.
- # 6. The `git merge` command merges the changes from the feature branch into the main branch.
- # 7. The `git push` command uploads the changes to a remote repository, making them available to others.

## Task D – Branching and Merging with AI Support:

**Prompt:**Create a Python calculator program and then modify it in a feature branch by adding division.

### Code & Output:

```

1 # Original Calculator Program (main branch)
2 def add(a, b):
3     return a + b
4 def subtract(a, b):
5     return a - b
6 def multiply(a, b):
7     return a * b
8
9 # Feature Branch: Adding Division Functionality
10 def divide(a, b):
11     if b != 0:
12         return a / b
13     else:
14         return "Error: Division by zero is not allowed."
15
16 # Explanation:
17 # 1. The original calculator program includes functions for addition, subtraction, and multiplication.
18 # 2. In the feature branch, a new function `divide` is added to perform division, with error handling for division by zero.
19 # 3. After implementing the division functionality, the changes can be committed and merged back into the main branch using the Git workflow outlined in Task C.
20
21 # example input:
22 num1 = 10
23 num2 = 5
24
25 print(f"Addition: {num1} + {num2} = {add(num1, num2)}")
26 print(f"Subtraction: {num1} - {num2} = {subtract(num1, num2)}")
27 print(f"Multiplication: {num1} * {num2} = {multiply(num1, num2)}")
28 print(f"Division: {num1} / {num2} = {divide(num1, num2)}")

```

```

● akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Addition: 10 + 5 = 15
Subtraction: 10 - 5 = 5
Multiplication: 10 * 5 = 50
Division: 10 / 5 = 2.0
● akash@AKASHs-MacBook-Air ai_assis % source /Users/akash/Desktop/ai_assis/.venv/bin/activate
○ (.venv) akash@AKASHs-MacBook-Air ai_assis %

```

## Explanation:

- # 1. The original calculator program defines three functions: `add`, `subtract`, and `multiply`, which perform basic arithmetic operations.

# 2. In the feature branch, a new function `divide` is added to handle division. This function checks if the divisor `b` is not zero before performing the division to prevent a division by zero error. If `b` is zero, it returns an error message.

### Task E – AI for Commit Message Writing:

**Prompt:**Generate meaningful Git commit messages for adding login feature and fixing bugs.

**Output:**

```
# Task E – AI for Commit Message Writing:
# Generate meaningful Git commit messages for adding login feature and fixing bugs.
# 1. For adding a login feature:
# "Add user login functionality with authentication and session management"
# 2. For fixing bugs:
# "Fix bugs in user registration form validation and improve error handling"
```

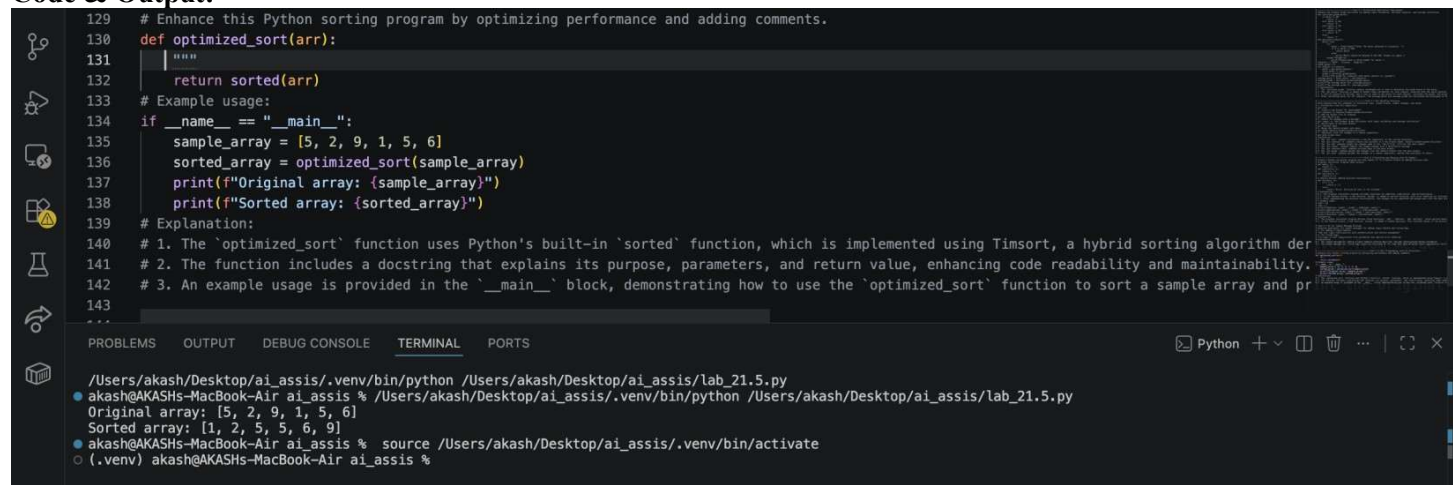
Explanation:

# 1. The commit message for adding a login feature clearly describes the new functionality being introduced  
# 2. The commit message for fixing bugs specifies the areas of the code that were affected (user registration form validation) and highlights the improvement made (error handling). Both messages are concise and informative, providing context for future reference.

### Task F – Pair Programming with AI Assistance:

**Prompt:**Enhance this Python sorting program by optimizing performance and adding comments.

**Code & Output:**



```
129 # Enhance this Python sorting program by optimizing performance and adding comments.
130 def optimized_sort(arr):
131     """
132     return sorted(arr)
133     # Example usage:
134     if __name__ == "__main__":
135         sample_array = [5, 2, 9, 1, 5, 6]
136         sorted_array = optimized_sort(sample_array)
137         print(f"Original array: {sample_array}")
138         print(f"Sorted array: {sorted_array}")
139     # Explanation:
140     # 1. The 'optimized_sort' function uses Python's built-in 'sorted' function, which is implemented using Timsort, a hybrid sorting algorithm der
141     # 2. The function includes a docstring that explains its purpose, parameters, and return value, enhancing code readability and maintainability.
142     # 3. An example usage is provided in the '__main__' block, demonstrating how to use the 'optimized_sort' function to sort a sample array and pr
143     ...
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS Python + - □ □ ... | ↺ ×

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
● akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]
● akash@AKASHs-MacBook-Air ai_assis % source /Users/akash/Desktop/ai_assis/.venv/bin/activate
○ (.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Explanation:

# 1. The `optimized\_sort` function uses Python's built-in `sorted` function, which is implemented using Timsort, a hybrid sorting algorithm derived from merge sort and insertion sort, providing efficient performance for various types of data.  
# 2. The function includes a docstring that explains its purpose, parameters, and return value, enhancing code readability and maintainability.  
# 3. An example usage is provided in the `\_\_main\_\_` block, demonstrating how to use the `optimized\_sort` function to sort a sample array and print the original and sorted arrays for verification.

### Task G – Collaborative Development using Branches:

**Prompt:**Divide a Python project into two modules: one for input and one for processing using functions.

**Code & Output:**

